MoCHII User Guide

Abstract

How to build and run MoCHII (MOnte Carlo for H II regions): requirements, the complete input-parameter reference, output-file formats, worked examples keyed to the regression tests, and the runbook for adding an ion. The physics and validation are documented separately in `docs/MoCHII_physics.pdf`; the atomic-data fitting pipeline in `docs/MoCHII_fitting.pdf`.

Contents

1. Requirements and build 1

2. Input parameters 2

2.1 Photons and RNG . . . 2
2.2 Grid and dust density . . . 3
2.3 Ionizing/FUV band and the source . . . 3
2.4 Initial state and iteration . . . 4
2.5 Thermal balance and metals . . . 5
2.6 I-front re-refinement (G4) . . . 5
2.7 Dust in the band and dust emission . . . 6
2.8 Peel-off imaging . . . 6
2.9 Output . . . 6

3. Output files 7

3.1 `<base>_rates.h5` . . . 7
3.2 Text outputs . . . 7
3.3 Reading outputs in Python . . . 7

4. Worked examples (the regression tests) 8

5. Adding an ion (the G5 runbook) 9

6. Practical notes 10

1. Requirements and build

- Intel oneAPI MPI Fortran (`mpiifort`/`mpiifx`; GNU `mpif90` supported by the Makefile), HDF5 (default prefix `/data/opt/hdf5_intel`; override with `make HDF5_PREFIX=...`), CFITSIO.
- The SEDust dust-emission library: `SEDust_lib/` in this tree holds `libsedust.a` and its `.mod` files (copied from the MoCafe v2.00 build). Only needed at link time; the SEDust *data* directory is referenced at run time (Section 2.7).

- Python 3 with numpy/scipy (h5py, astropy, matplotlib, PyNeb for the test and pipeline scripts).

Build:

```
cd MoCHII
make           # -> MoCHII.x   (always a full clean rebuild)
make DEBUG=1   # bounds/traceback build
```

The Makefile intentionally performs a full rebuild every time (the MoCafe policy): there is no inter-module dependency tracking, and stale `.mod` files after editing `define.f90` otherwise cause runtime namelist errors.
Run:

```
mpirun -np 8 ./MoCHII.x input.in
```

MoCHII is MPI-only; the octree and all leaf arrays live in MPI-3 shared memory (one copy on each node).

2. Input parameters

All input is one `&parameters` namelist assigning fields of `par`. Parameters not listed keep the defaults shown. Stage switches compose: `gas_niter = 0` gives a G0 single pass (rates only); adding `gas_niter > 0` iterates the ionization (G1); `solve_te` adds the thermal balance (G2); `use_metals` the trace-metal registry; `diffuse_field` the explicit diffuse field (G3, forces case A); `refine_front` the I-front re-refinement (G4); `ion_add_dust` the dust coupling.

2.1 Photons and RNG

parameter	default	meaning
`no_photons`	1e5	source packets in each iteration
`no_print`	1e7	progress-print stride
`iseed`	0	RNG seed (0 = from clock/pid)

2.2 Grid and dust density

parameter	default	meaning
`grid_type`	'car'	`'amr'` (octree) or `'uniform'` (single-level leaf list stored in raster order; traversal by integer-stepping DDA, no tree or neighbor table in memory; `refine_front` unavailable)
`amr_file`	"	generic AMR leaf file (FITS/HDF5/text): columns x, y, z, level, nH [, metallicity, xHI, ndust]
`distance_unit`	'kpc'	'kpc'/'pc'/'au'/" (code units)
`dust_model`	'global_dgr'	leaf grey opacity: `'none'`, `'global_dgr'` ($n_{\rm H}c_{\rm ext}$·DGR), `'from_file'` (ndust column), `'laursen09'` (static xHI column), `'laursen09_live'` (computed $x_{\rm HI}$, refreshed each iteration)
`cext_dust`	1.6059e-21	reference extinction per H [cm^2] at `lambda_ref`
`DGR`, `Z_global`, `Z_ref`	1e-2, 0.0134, 0.0134	dust-model scalings
`f_ion_dust`	0.01	laursen09 survival of dust in ionized gas
`f_pah`, `f_ion_pah`	0, 0	PAH share of the reference extinction and its own ionized-gas survival (laursen09_live)

2.3 Ionizing/FUV band and the source

parameter	default	meaning
`use_ion_band`	F	must be T (the MoCHII transport)
`nnu_ion`	16	log-spaced bins; gates use 32; raise when widening the band
`eion_min`, `eion_max`	13.598, 100	ionizing band edges [eV]
`add_fuv`	F	FUV extension: extra log bins on [`efuv_min`, `eion_min`] below the unchanged ionizing bins; transports FUV grain heating and FUV photoionization of low-threshold metals (Mg I, C I, S I, Fe I); `luminosity` stays the IONIZING luminosity ($Q_{\rm H}$ preserved, the FUV part rides on top)
`efuv_min`	6.0	FUV lower edge [eV]
`nnu_fuv`	8	FUV bins (16 recommended with metals)
`tstar`	−999	Planck source temperature [K]
`ion_spectrum`	"	alternative 2-column spectrum file (E [eV], L_E arb.)
`luminosity`	1.0	BAND luminosity [erg/s]
`xs_point`, `ys_point`, `zs_point`	0	point-source position (code units, recentered box)

2.4 Initial state and iteration

parameter	default	meaning
`xHI_init`	1.0	initial $n_{\mathrm{HI}}/n_{\mathrm{H}}$ (an `xHI` file column wins); an ionized start (1e-3) converges much faster than neutral
`xHeI_init`, `xHeII_init`	1, 0	initial He fractions
`He_abund`	0.1	$n_{\mathrm{He}}/n_{\mathrm{H}}$
`gas_niter`	0	iterations (0 = G0 rates-only pass)
`gas_tol`	1e-3	convergence tolerance on x_{HII} (measure set by `conv_crit`)
`conv_crit`	'cell'	'cell': cell-max tests ($\max\lvert\Delta x_{\mathrm{HII}}\rvert$ — stalls at the front-cell Monte Carlo noise floor $\sim$2–4e-2 at 4e7 packets); 'vol': volume-integrated tests $\sum V\lvert\Delta x_{\mathrm{HII}}\rvert/\sum V x_{\mathrm{HII}} <$ `gas_tol` and $\sum V n_e\lvert\Delta T_e\rvert/\sum V n_e T_e <$ `gas_tol_te` (recommended; noise-robust, and the n_e weight removes vacuum and no-heating cells). The vol measures floor at their own Monte Carlo level ($\sim$3e-3 and 7e-3 at 8e6 packets, scaling as $1/\sqrt{N}$) — set the tolerances just above it
`ion_relax`	1.0	under-relaxation on x (< 1 damps front oscillation)
`case_ab`	'B'	case A/B recombination ('A' forced by the diffuse field)

2.5 Thermal balance and metals

parameter	default	meaning
`solve_te`	F	bisection on T_e coupled with the ionization
`te_fixed`	1e4	fixed T_e [K] when `solve_te` is off
`te_min`, `te_max`	1e3, 5e4	bisection bracket [K]
`gas_tol_te`	1e-3	convergence on max $\|\Delta T_e\|/T_e$
`atomic_dir`	'data/atomic'	coefficient files (relative to the run directory)
`use_metals`	F	registry cascade + Tier-1 metal cooling
`abund_C/N/O/Ne/S`	HII20 values	$n(\mathrm{X})/n(\mathrm{H})$; an element is active when > 0
`abund_Ar/Mg/Fe`	0	further registry elements (gas-phase values; Mg/Fe are heavily depleted)
`ion_metal_abs`	T	metal photoionization absorption in the band opacity (active with `use_metals`); the only gas opacity in the FUV bins
`emis_output`	F	write `_emis` (HDF5/FITS): leaf-by-leaf line emissivities + the state needed for maps (Section 3.)
`diffuse_field`	F	explicit ground-recombination packets (G3)
`metal_ne`	F	PDR: electrons from the metal cascade join the n_e closure (the only electrons beyond the I-front)
`metal_heat`	F	PDR: metal photoheating in the thermal balance
`grain_pe`	F	PDR thermal package: grain photoelectric heating + grain recombination cooling (Bakes & Tielens 1994) from the local FUV field (needs `add_fuv`), plus H-impact [C II] 158 μm / [O I] 63 μm cooling; grain terms scaled by `pe_scale` $\times$ the leaf dust amount relative to MW (from the leaf opacity)
`pe_scale`	1.0	photoelectric-efficiency scale factor

2.6 I-front re-refinement (G4)

parameter	default	meaning
`refine_front`	F	rebuild the octree on the I-front once
`refine_iter`	20	at which iteration
`refine_lbase`, `refine_lmax`	5, 7	base/front levels
`refine_eps`	1e-2	front flag: $\epsilon < x_{\mathrm{HI}} < 1-\epsilon$
`refine_dx`	0.2	or a face-neighbor x_{HI} jump above this

2.7 Dust in the band and dust emission

parameter	default	meaning
`ion_add_dust`	F	dust absorption in the band (dusty Strömgren)
`ion_dust_kext`	”	kext table with EUV coverage (e.g. `data/kext_albedo_WD_MW_3.1_60_D03.all_2009`); required
`ion_dust_scatter`	F	sample dust scattering (HG, albedo weights)
`dust_sed`	F	SEDust stochastic dust SED at output time
`dust_model_sed`	’astrodust’	’astrodust’/’dl07’/’zubko’
`sed_workdir`	”	a SEDust `sed/` tree (dielectric tables are read relative to it), e.g. the MoCafe copy `.../MoCafe_v2.00/SEDust/sed`
`sed_qtable`, `sed_sizedist`	MoCafe defaults	optics/size-distribution tables (relative to `sed_workdir`)
`sed_NT`, `sed_Tlo`, `sed_Thi`	200, 2.7, 5e3	SEDust temperature grid
`dust_emis_bands`	unset	up to 8 wavelengths [μm] (e.g. 8, 24, 70, 160): SEDust band emissivities of each leaf $\rightarrow$ `_dustemis` (blocks `emis_dust/wl_dust` [erg s^{-1} cm^{-3} μm^{-1}]); the Python map maker projects them into dust-band images (the IR is optically thin)

2.8 Peel-off imaging

parameter	default	meaning
`ion_peel`	F	after convergence, one extra transport pass peels direct (stellar + diffuse) and dust-scattered contributions toward the observers $\rightarrow$ `_image` (blocks `direct_ion/scatt_ion`, plus `_fuv` with `add_fuv`) [erg s^{-1} cm^{-2} sr^{-1}]
`obsx,obsy,obsz`	unset	observer positions (or directions) in code units; or use `alpha/beta/gamma` angle lists [deg]; default: one observer on +z far away
`distance`	auto	observer distance (code units)
`nxim`, `nyim`	129, 129	image pixels
`dxim`, `dyim`	auto	pixel scale [deg]; auto-sized to cover the box

2.9 Output

`out_file` (base name; extension follows `file_format` = ’hdf5’ or ’fits’).

3. Output files

3.1 <base>_rates.h5

One dataset for each extension, leaf-ordered:

extension	content
Gamma_HI/HeI/HeII	photoionization rates [s^{-1}]
Heat_HI/HeI/HeII	photoheating per particle [erg/s]
Heat_dust	grain heating [erg s^{-1} cm^{-3}] (when ion_add_dust)
T_dust	equilibrium mixture dust temperature [K]
x_HI, x_HeI, x_HeII, n_e, T_e	converged state
x_<el>_stages	metal ion fractions (stage, leaf)
J_nu	band mean intensity (bin, leaf) [erg/s/cm^2/Hz/sr]
E_bin, dE_bin	band grid [eV]
LeafXYZ	leaf centers (code units)

3.2 Text outputs

file	content
_lines.txt	line luminosities: SH95 H I recombination lines + Tier-2 collisional lines; columns elem, stage, λ[Å], L, $L/L(\mathrm{H}\beta)$
_emis.h5	(with emis_output) leaf-by-leaf line emissivities: emis_h (SH95 lines) and emis_<el>_<stage> blocks [erg s^{-1} cm^{-3}; $\sum \mathrm{emis} \cdot V = L$, rows match _lines.txt], wl_* wavelengths [Å], plus LeafXYZ, LeafSize, n_H, n_e, T_e, x_HI/HeI/HeII, x_<el>_stages — self-contained for line maps
_nebcont.txt	nebular continuum L_ν (fb+ff tables, Milne, two-photon)
_dustir.txt	equilibrium-mixture dust IR spectrum (1–1000 μm) with the energy-closure header
_dustsed.txt	SEDust stochastic dust SED with PAH bands (when dust_sed)
_dustemis.h5	(with dust_emis_bands) SEDust band emissivities of each leaf, same block layout as _emis — readable by EmisData for dust-band maps
_image.h5	(with ion_peel) peel-off images for each observer: direct_ion/scatt_ion (+_fuv) [erg s^{-1} cm^{-2} sr^{-1}], suffix _obs<k> when several observers

3.3 Reading outputs in Python

tools/python/mochii_output.py is an importable module (works directly in a notebook) that reads every section file (HDF5 or FITS; read_sections), the line table (read_lines_table), and the text spectra (read_spectrum), and builds 2D maps from the emissivity file:

```
import sys; sys.path.insert(0, ".../MoCHII/tools/python")
```

```
import matplotlib.pyplot as plt
from mochii_output import EmisData

d = EmisData("run_emis.h5")
d.info()                              # blocks, fields, line count
d.line_list()                         # every stored line (block, wl)

# ready-made figures (notebook): returns (fig, ax)
fig, ax = d.plot_line("o_3", 5007.)                  # log S map
fig, ax = d.plot_line("h", 6563., unit="intensity",
                      photons=True)  # photons/s/cm^2/sr
fig, ax = d.plot_field("T_e", weight="EM")
fig, axs = plt.subplots(1, 2); d.plot_line("s_2", 6717., ax=axs[0]) \
    ; d.plot_line("s_2", 6731., ax=axs[1])   # doublet side by side

# raw arrays, when the figure is yours to build
img, ext = d.line_map("o_3", 5007., axis="z", npix=512)
img, ext = d.line_map("h", 6563., unit="intensity", photons=True)
img, ext = d.field_map("T_e", weight="EM")   # LOS-weighted average
Q = d.line_luminosity("h", 4861., photons=True)  # photon rate [1/s]
```

Line maps are flux-conserving: each leaf's luminosity is deposited with exact area overlap onto the pixel grid (AMR leaf sizes handled), so $\sum SA_{\rm pix} = L_{\rm line}$ to machine precision. `unit='flux'` (default) gives surface brightness [erg s^{-1} cm^{-2}]; `unit='intensity'` gives specific intensity [erg s^{-1} cm^{-2} sr^{-1}] $= S/4\pi$ (isotropic emission, optically thin); `photons=True` converts either to photon units (divides by hc/λ of the line). A `slab=(lo,hi)` cut restricts the line of sight. Field maps average `T_e`, `n_e`, `x_HI`, … along the line of sight with weight `'EM'` ($n_e^2 V$), `'ne'`, `'nH'`, or `'V'`. The same module doubles as a command-line tool for quick looks:

```
python3 mochii_output.py run_emis.h5                  # list lines
python3 mochii_output.py run_emis.h5 --line o_3 5007 \
        --npix 512 --log --out o3.png
python3 mochii_output.py run_emis.h5 --line h 6563 \
        --unit intensity --photons     # photons/s/cm^2/sr
python3 mochii_output.py run_emis.h5 --field T_e --weight EM
```

4. Worked examples (the regression tests)

Each `tests/` directory is self-contained: grid builder or a symlinked grid, the input file, and a `check_*.py` gate script.

directory	what it demonstrates / gate
`g0_gamma`	rate integrals vs. analytic attenuation (bias +0.004%)
`g1_stromgren`	Strömgren sphere, case B fixed T_e; AMR $\equiv$ uniform
`g2a_thermal`	thermal H/He nebula vs. the 1D exact reference
`g2_hii`	MOCASSIN Lexington HII20/HII40 benchmarks + line fluxes
`g4_refine`	I-front re-refinement vs. native level 7
`g4_tng`	Illustris-TNG post-processing demo (shadows, maps)
`d_dusty`	dusty Strömgren; scattering bracket; T_d/IR

A minimal thermal + metals + dust input (from `d_dusty`):

```
&parameters
 par%no_photons = 4.0e7           par%iseed = 1234
 par%grid_type = 'amr'            par%amr_file = 'amr_L6.fits'
 par%distance_unit = 'pc'         par%dust_model = 'global_dgr'
 par%use_ion_band = .true.        par%nnu_ion = 32
 par%eion_min = 13.598            par%eion_max = 100.0
 par%tstar = 4.0e4                par%luminosity = 3.178e38
 par%xHI_init = 1.0e-3            par%He_abund = 0.1
 par%gas_niter = 60               par%gas_tol = 2.0e-3
 par%solve_te = .true.            par%atomic_dir = '../../data/atomic'
 par%use_metals = .true.
 par%ion_add_dust = .true.
 par%ion_dust_kext = '../../data/kext_albedo_WD_MW_3.1_60_D03.all_2009'
 par%cext_dust = 4.868e-22        par%DGR = 1.0
 par%out_file = 'run.h5'          par%file_format = 'hdf5'
/
```

5. Adding an ion (the G5 runbook)

Adding an ion touches only data:

1. `tools/fitting/fit_cooling_tier1.py`: add the ion with a T_i ladder guessed from its `.elvlc` level energies; run; check the reported max error and the overlay figure.

2. `tools/fitting/fit_nlevel_tier2.py`: add the ion with the level count needed for its diagnostics; run.

3. `tools/fitting/make_element_data.py`: add the element to `ELEMENTS` (Z, stage count) and its thresholds; charge exchange entries where sources exist; run. The CHIANTI `.rrparams`/`.drparams` types 1–3 are handled (`RR`/`RR2`/`DR`/`DR2` lines).

4. Fortran: a `par%abund_<El>` field and one entry in the `species_setup` name/abundance lists (only for a NEW element; new stages of a known element need nothing).

5. Verify: extend `verify_nlevel_pyneb.py` with a diagnostic ratio where PyNeb has data; run a smoke test and check the new lines in `_lines.txt`.

Data caveats met so far: Ar II has no CHIANTI v11 level/collision data (stage tracked without lines); Fe III collision strengths differ from PyNeb's BB14 by up to ∼16% in 4658/4702 (known spread for iron); charge-exchange stage labels differ between the Huang (2023) and MOCASSIN transcriptions for Mg/Fe (the Huang labels are adopted; see `make_element_data.py`).

6. Practical notes

- **Bins**: 32 bins give ∼2% rate-integral discretization; use 64 for sub-percent work. The `add_fuv` extension carries its own bins below `eion_min`, so the ionizing resolution is unaffected.
- **FUV and the PDR**: without `add_fuv` there is no radiation beyond the I-front and the metals there recombine toward neutral; with it, FUV penetrates the neutral gas (dust extinction only) and produces the PDR-side C II and Mg II. Metal photoheating and grain photoelectric heating are not in the thermal balance, so PDR-zone temperatures are indicative only.
- **Convergence**: use `conv_crit = 'vol'` for production — the cell-max criterion stalls at the front-cell noise floor (and, with `add_fuv`, at no-heating cells where T_e pins to `te_min`). Both measures are printed each iteration. The G3 diffuse field smooths the front and converges far better.
- **Starts**: prefer an ionized initial state; a neutral start advances the front ∼one cell per iteration.
- **kext tables**: must cover the band (the astrodust table stops at 912 Å; use the D03 table for EUV work). Tables are sorted on read (the D03 file has an out-of-order seam).
- **Re-refinement**: one event per run; the old shared-memory windows are reclaimed only at exit.
- **SEDust**: `sed_workdir` must point at a SEDust `sed/` tree; the exact stochastic solve costs ∼0.05–0.1 s for each gas leaf (distributed over ranks).